Inserting Fmap with Coercive Subtyping

This paper shows how to use subtyping to infer uses of the basic monad operations.

CCS Concepts: • **Software and its engineering → Functional languages**.

Additional Key Words and Phrases: subtyping, functors

ACM Reference Format:
. 2026. Inserting Fmap with Coercive Subtyping. *J. ACM* 37, 4, Article 111 (August 2026), 3 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Typeclass abstractions like Functor, Applicative, and Monad are essential to modern functional programming. The liberal application of type class combinators allows concise programming even in the presence of complex effects and data structures. For example, consider writing a Haskell program that increments integers wrapped in a `Maybe` type.

```
incrMaybe : Maybe Int -> Maybe Int
incrMaybe (Just x) = Just (succ x)
incrMaybe Nothing  = Nothing
```

This naive definition of `incrMaybe` has to unwrap its argument with a pattern match and then wrap its output back up. A more idiomatic definition uses <$>, the fmap combinator, which handles the unwrapping and wrapping.

```
incrMaybe x = succ <$> x
```

In the presence of sufficient type information, we are interested in further reducing the boilerplate involved by making fmap implicit. This permits writing `incrMaybe` with no use of fmap at all.

```
incrMaybe x = succ x
```

It seems as if there is only one canonical way to make this program make sense, and automatically lifting functions to operate on arrays and other data structures is already prevalent both in programming languages and mathematical notation.

In this paper, we study a *coercive subtyping* discipline that infers where to insert implicit fmaps using subtyping. Each derivation of $A <: B$ then corresponds to a coercion $A \to B$ expressed in a simple combinator calculus. In coercive subtyping, type inference is coupled to an elaboration pass, which inserts coercions in the program based on the results of subtyping queries. This makes the operational meaning of surface programs depend on the result of type inference, which breaks with a more naive view of inference as merely tacking missing type annotations on to a program with already fixed semantics. Indeed, many programming languages confound their users with strange behavior involving implicit coercions, whether they are inserted statically or dynamically.

We can avoid these confusions by establishing *coherence*, a basic metatheoretic property of systems of coercions. This problem was studied as early as 1980 by Reynolds [1], who outlined an

Author's Contact Information:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-735X/2026/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

$$A ::= \mathsf{bool} \mid A \to A \mid \mathsf{F}\,A$$

$$\textsc{Refl}\ \frac{}{A \le A} \qquad \textsc{Trans}\ \frac{A \le B \qquad B \le C}{A \le C} \qquad \textsc{Arr}\ \frac{U \le A \qquad B \le V}{A \to B \le U \to V}$$

$$\textsc{Mono}\ \frac{A \le B}{\mathsf{F}\,A \le \mathsf{F}\,B} \qquad \textsc{Fmap}\ \frac{}{A \to B \le \mathsf{F}\,A \to \mathsf{F}\,B}$$

Fig. 1. Types and subtype relationships for a calculus with implicit fmap.

algebraic approach to proving the coherence of coercions. In his view, coercions form a *category*: there are types $A, B, \ldots$ and between any two types there is a set of coercions $\mathrm{Coe}(A, B)$. There are always identity coercions $\mathsf{id}_A \in \mathrm{Coe}(A, A)$, which do nothing, and there are associative and unital composition operations $\circ : \mathrm{Coe}(B, C) \times \mathrm{Coe}(A, B) \to \mathrm{Coe}(A, C)$. The coercions are expected to form a preorder, that is, for every pair of arrows $c1, c2 \in \mathrm{Coe}(A, B)$, $c1 \equiv c2$. He also studies their interaction with generic operators of the language, but we will restrict our attention to the coercions themselves.

As is typical in programming languages, we establish our coercions are coherent by showing every coercion $c : A \le B$ can be put into a normal form $\mathsf{nf}\,c$ that is equivalent to c, and then show that normal forms are unique. Rather than relying on rewriting, we adopt the technique of *normalization by evaluation* (NbE).

Besides giving some confidence in the safety of this programming model, the proof of soundness has independent interest as a case study in the application of normalization by evaluation to implicit coercions.

2 Overview

The types and subtyping rules at issue are shown in Figure 1. We call these the declarative rules (as opposed to the algorithmic rules). The only nonstandard rule is Fmap, which permits using functions $A \to B$ at the lifted type $\mathsf{F}\,A \to \mathsf{F}\,B$. This rule may be iterated, permitting the use of functions at $\mathsf{F}\,A \to \mathsf{F}\,B$, $\mathsf{F}\,\mathsf{F}\,A \to \mathsf{F}\,\mathsf{F}\,B$, and so on. These rules are more or less evident, but they have practical and theoretical limitations.

The declarative rules are not suitable for an implementation because they are not deterministic: too many rules apply to possible subtyping queries. Besides the fact that it is always possible to appeal to Trans, queries of the form $A \to B \le \mathsf{F}\,U \to \mathsf{F}\,V$ must choose between Arr and Fmap.

They are also questionable from a semantic point of view because a single judgment may have many derivations and the coercions associated with these derivations may not be equivalent.

3 Coercions

Grammar of c

4 Normal Coercions

Grammar of n, ne

5 Soundness and Completeness

Discussion of the NbE method.

Discussion of the proof.

6 Implementation

Using Chronolog, we did some examples.

7 Related Work

Reynolds?

8 Conclusion

References

[1] John C. Reynolds. 1980. Using category theory to design implicit conversions and generic operators. In *Semantics-Directed Compiler Generation (Lecture Notes in Computer Science)*. Springer, 211–258.